AI Agent Developer

Complete Interview Preparation Topic Checklist

A practical roadmap focused on concepts, system workflows, troubleshooting, debugging, and explaining existing systems — with coding depth prioritized according to interview needs.

Target roles: AI Agent Developer, Generative AI Engineer, LLM Application Developer, Junior AI Engineer

Use this as a checklist. Learn the highest-priority topics first, then practise explaining how systems work and how you would diagnose failures.

How to use this guide

The goal is to prepare for interviews in a practical way. Focus on understanding concepts, tracing workflows, reading existing code, identifying root causes, and making small safe changes. Interviews can still include Python, SQL, code-reading, or debugging exercises, so basic hands-on ability remains important.

Priority levels

- Tier 1 — Master first: Python/debugging, REST APIs/JSON, LLM fundamentals, embeddings, RAG, vector databases, tool calling, agent architecture, troubleshooting.
- Tier 2 — Prepare next: LangGraph/LangChain, agent memory, MCP, evaluation, testing, security, SQL, FastAPI, project architecture.
- Tier 3 — Role-dependent: Docker, cloud deployment, advanced multi-agent systems, fine-tuning, Graph RAG, and model optimization.

1. Python fundamentals for AI developers

- Variables, data types, and operators
- Lists, tuples, sets, and dictionaries
- Conditions and loops
- Functions, arguments, and return values
- Mutable vs immutable objects
- List and dictionary comprehensions
- String manipulation
- Exception handling: try, except, finally
- Classes and objects; basic object-oriented programming
- Modules, packages, and imports
- File handling: CSV, JSON, and TXT
- None, True, and False
- Shallow copy vs deep copy
- Iterators and generators — basics
- async and await — basic understanding
- Reading stack traces and debugging errors
- Virtual environments, pip, and dependencies
- Environment variables and .env files
- Reading existing Python code and tracing execution

Interview note: Common questions: list vs tuple, what a dictionary is, exception handling, and debugging KeyError, ImportError, or TypeError.

2. REST APIs, HTTP, and JSON

- What an API is and what REST means
- Client-server architecture
- Endpoints, resources, URLs, and query parameters
- HTTP methods: GET, POST, PUT, PATCH, DELETE
- Request body vs response body
- HTTP status codes: 200, 201, 400, 401, 403, 404, 429, 500, 502, 503

- Headers, authentication, and content types
- JSON objects, arrays, keys, and values
- API keys, bearer tokens, and OAuth basics
- Rate limits, pagination, and timeouts
- Retries, exponential backoff, and error handling
- Synchronous vs asynchronous requests
- Webhooks and API integrations
- Testing APIs with Postman or Python
- Reading API documentation
- Diagnosing failed API calls from logs and responses

Interview note: Troubleshooting example: for HTTP 429, inspect rate limits, quota, request frequency, retry behavior, billing, and access restrictions.

3. Git, GitHub, and development tools

- Git and GitHub; repository, commit, branch, and merge
- clone, pull, add, commit, and push
- Pull requests and merge conflicts
- .gitignore, README files, and project documentation
- Managing Python dependencies
- VS Code and Jupyter Notebook basics
- Terminal commands and folder navigation
- Debugging with logs, breakpoints, and error messages
- Understanding project structure
- Why API keys must not be committed to GitHub

4. SQL and databases

- Tables, rows, columns, primary keys, and foreign keys
- SELECT, WHERE, ORDER BY, GROUP BY
- INNER JOIN and LEFT JOIN
- Aggregate functions and subqueries
- INSERT, UPDATE, and DELETE
- NULL values and duplicate records
- Indexes and basic query performance
- Relational databases vs vector databases
- Connecting Python applications to databases
- Parameterized queries and SQL injection prevention
- Transactions and read-only access
- Troubleshooting database connection failures
- Understanding database errors and slow queries

5. Machine learning and NLP fundamentals

- AI vs machine learning vs deep learning vs generative AI

- Supervised, unsupervised, and reinforcement learning
- Training vs inference
- Features, labels, models, and predictions
- Classification, regression, and clustering
- Overfitting and underfitting
- Training, validation, and test datasets
- Precision, recall, F1 score, and accuracy
- Natural language processing (NLP)
- Tokens, tokenization, and vocabulary
- Transformers and attention — conceptual understanding
- Neural networks and embeddings — basics
- Semantic similarity and cosine similarity
- Model evaluation and limitations

6. LLM fundamentals

- What a large language model is and how it generates responses
- Transformer architecture — high-level understanding
- Tokens and context windows
- System prompts, user prompts, and conversation history
- Temperature and output randomness
- Maximum output tokens
- Hallucinations and why they occur
- Context limitations and knowledge cutoffs
- Hosted models vs open-weight models
- Model selection based on cost, latency, and capability
- Structured outputs and JSON mode
- Function calling / tool calling
- Streaming responses
- Model inference and API errors
- LLM latency, token usage, and rate limits
- Prompt engineering and few-shot examples
- Why LLM outputs are not always deterministic

Interview note: Common question: why do LLMs hallucinate, and how can RAG, grounding, validation, and evaluation reduce the problem?

7. Embeddings and vector databases

- What an embedding is
- Text embeddings vs generated text
- Semantic similarity and cosine similarity
- Embedding dimensions and choosing an embedding model
- Why query and document embedding models must be compatible
- Vector indexes and nearest-neighbor search

- FAISS vs Chroma vs managed vector databases
- Collections, IDs, metadata, and document updates
- Top-k retrieval and similarity thresholds
- Metadata filtering
- Approximate nearest-neighbor search — basics
- Embedding storage and indexing failures
- Vector database persistence
- Rebuilding an index after changing the embedding model

Interview note: Troubleshooting: if retrieval quality drops after changing embedding models, verify compatibility and re-embed documents before rebuilding the index.

8. RAG — Retrieval-Augmented Generation

- RAG fundamentals; why RAG is used
- RAG vs fine-tuning vs prompt engineering
- RAG architecture and end-to-end workflow
- Knowledge bases and document ingestion
- PDF, CSV, JSON, HTML, DOCX, and Markdown ingestion
- Text extraction and document cleaning
- Chunking strategies, chunk size, and overlap
- Metadata and source tracking
- Embedding generation and vector indexing
- Retrieval and context construction
- Grounded response generation
- Source citations and evidence verification
- Keyword search vs semantic search
- BM25, dense retrieval, and hybrid search
- Reranking, top-k, and similarity thresholds
- Query rewriting, multi-query retrieval, and metadata filtering
- Handling long documents and tables
- Parent-child retrieval — conceptual understanding
- Missing or incorrect PDF text extraction
- Poor chunking and lost context
- Relevant documents not retrieved or irrelevant documents returned
- Embedding dimension mismatch and vector database failures
- Stale or duplicated indexed documents
- Correct context retrieved but wrong answer generated
- Unsupported answers, hallucinations, and missing citations
- Slow retrieval and excessive context
- Outdated knowledge-base information
- RAG evaluation: Precision@k, Recall@k, MRR, faithfulness, relevance, context precision
- Ground-truth question-answer datasets and regression testing

Interview note: Common question: if the right information exists but the answer is wrong, inspect ingestion, chunking, embeddings, retrieval, context construction, generation, and evaluation separately.

9. AI agent fundamentals

- What an AI agent is
- Chatbot vs LLM workflow vs AI agent
- Agent architecture and execution loop
- Reasoning, planning, acting, and observing
- Goals, tasks, and intermediate steps
- Autonomous agents vs deterministic workflows
- State and context management
- Tool selection and execution
- Agent memory
- Short-term vs long-term memory
- Task decomposition and conditional routing
- Stop conditions and iteration limits
- Reflection and verification patterns
- Agent failure recovery
- Human-in-the-loop workflows
- When an agent is unnecessary
- Reliability challenges in autonomous systems
- ReAct, planner-executor, router, supervisor-worker, reflection, and stateful workflow patterns

10. Function calling and tools

- What function calling is and how an LLM selects a tool
- Tool definitions, descriptions, and schemas
- Arguments and structured parameters
- Tool execution and result handling
- Tool calling vs normal text generation
- Multiple tool calls
- Tool validation
- Invalid arguments and execution errors
- Tool permissions and authorization
- Idempotency and duplicate actions
- Retry policies and timeouts
- Read-only vs write operations
- Human approval before sensitive actions
- API integrations, database queries, search, calculator, and document retrieval as tools

Interview note: Scenario: if the agent chooses the correct tool but supplies bad arguments, inspect the schema, validate inputs, handle model output, retry safely, and log the failure.

11. Agent memory and state

- Conversation history and session state
- Working memory and persistent memory
- Summarizing long conversations
- Retrieving relevant memories
- Storing state in databases
- Checkpointing and resuming workflows
- State transitions and context-window limits
- Memory relevance and stale information
- Privacy, access control, and deletion
- Distinguishing retrieved knowledge from user-specific memory

12. Agent frameworks

- LangChain: models, prompts, messages, tools, retrievers, loaders, chains, parsers, integrations
- LangGraph: graphs, nodes, edges, state schemas, conditional routing, cycles, termination, checkpoints, persistence, interrupts, human approval, recovery
- LlamaIndex: ingestion, indexes, retrievers, query engines, data connectors, RAG pipelines, agent tools
- CrewAI: agents, roles, tasks, and coordination — awareness
- AutoGen: conversational agent workflows — awareness
- Pydantic AI: typed inputs, outputs, and dependencies — awareness
- Understand the problem each framework solves without trying to master all frameworks

13. MCP — Model Context Protocol

- What MCP is and why it exists
- MCP host, client, and server
- Tools, resources, and prompts
- How an agent discovers and invokes MCP tools
- MCP vs REST APIs vs function calling
- Local and remote MCP servers
- Transport and connection basics
- Authentication and permissions
- Tool descriptions and input schemas
- MCP connection failures and debugging
- Security risks from untrusted MCP servers
- Approval controls and least privilege

Interview note: Common question: MCP standardizes tool discovery and access across compatible applications; it does not replace every API.

14. Multi-agent systems

- Single-agent vs multi-agent architecture
- Why split work across agents
- Supervisor and worker agents; specialist agents

- Delegation and handoffs
- Shared state vs isolated state
- Communication between agents
- Result aggregation and verification
- Conflicting outputs
- Infinite loops and runaway costs
- Coordination failures and timeouts
- When multi-agent systems add unnecessary complexity

15. LLM evaluation and observability

- Unit testing vs integration testing
- Golden datasets and expected answers
- Evaluating retrieval, generation, and tool calls separately
- LLM-as-a-judge and human evaluation
- Faithfulness, relevance, and correctness
- Task completion rate
- Tool selection and argument accuracy
- Latency and token consumption
- Logging, traces, and execution history
- LangSmith and OpenTelemetry — purpose and basics
- Reproducing intermittent failures
- Regression tests after prompt or model changes
- Monitoring production quality
- Diagnosing model, prompt, retrieval, and infrastructure failures

Interview note: Key principle: a successful API response does not necessarily mean the agent completed the task correctly.

16. AI safety and security

- Prompt injection and indirect prompt injection
- Jailbreaks and instruction conflicts
- Untrusted input and retrieved content
- Sensitive data exposure, PII handling, and privacy
- Authentication vs authorization
- API key and secret management
- Least-privilege tool access
- SQL injection and unsafe tool inputs
- Data exfiltration risks
- Tool allowlists and human approval for consequential actions
- Output validation and sandboxing
- Audit logs, denial-of-service, and excessive tool loops
- Secure handling of third-party integrations

17. Backend and application architecture

- FastAPI fundamentals and REST endpoint design
- Request and response validation; Pydantic models
- Authentication and authorization
- HTTP status codes
- Async request handling and streaming responses
- Background tasks
- Database connectivity and error handling
- Logging, session management, and caching basics
- Service-to-service communication
- Rate limiting and API testing
- Separating frontend, backend, model, and retrieval layers

18. Docker, cloud, and deployment

- Development vs staging vs production
- Docker images and containers
- Dockerfiles and environment configuration
- Ports, volumes, and networking
- Dependency and version conflicts
- Deploying a FastAPI application
- Cloud compute and storage basics
- Environment variables and secret stores
- IAM and access permissions
- Managed model services and managed vector databases
- Health checks, monitoring, and CI/CD basics
- Logs and production debugging
- Scaling, latency, cost management, rollback, and recovery
- Learn one cloud platform deeply enough for target roles; Azure, AWS, or GCP

19. Fine-tuning and model optimization

- RAG vs fine-tuning vs prompt engineering
- When fine-tuning is appropriate
- Supervised fine-tuning basics
- LoRA and QLoRA concepts
- Quantization
- Model size vs latency
- GPU and memory constraints
- Caching and batching
- Model routing
- Selecting models by task and cost
- Evaluating model changes

- Open-weight model licensing

Interview note: Secondary priority: understand when fine-tuning helps; you generally do not need to implement a fine-tuning pipeline for entry-level agent roles.

20. Troubleshooting scenarios to practise

- LLM API returns an error: inspect status code, response body, credentials, model access, quota, rate limits, schema, timeout, and provider status
- RAG returns irrelevant documents: inspect extracted text, chunks, query embedding, stored embeddings, filters, top-k, scores, and reranking
- Correct documents but wrong answer: inspect assembled context, prompt, ordering, model behavior, conflicting sources, and citations
- Agent calls the wrong tool: review tool descriptions, available tools, routing, argument schemas, and selection output
- Agent gets stuck in a loop: inspect state transitions, repeated results, retries, stop conditions, recursion limits, and iteration caps
- Works locally but fails after deployment: compare environment variables, dependencies, network, permissions, paths, resources, and configuration
- Application is slow: measure latency per API, retrieval, reranking, database, tool, and generation stage; inspect retries, concurrency, and context size
- Agent falsely reports success: verify actual tool results and side effects before returning success
- Duplicate actions: check retries, repeated tool calls, idempotency keys, state persistence, and ambiguous outcomes
- Sensitive information appears: investigate retrieval permissions, access controls, logs, injection, output filtering, and tool authorization
- Invalid JSON: inspect schema, parsing, required fields, response format, validation, and controlled retries
- Outdated knowledge base: check document versions, metadata, refresh jobs, deletion handling, indexing status, and source of truth

21. Project and resume interview questions

- What problem does the project solve?
- Why choose RAG or an agent instead of a simpler solution?
- Explain the architecture
- What data sources were used and what are their licenses?
- How was data cleaned and processed?
- Why choose that embedding model and vector database?
- How does retrieval work?
- What happens when the answer is absent from the knowledge base?
- How was accuracy measured?
- What failures occurred and how were they fixed?
- What security controls were implemented?
- How are latency and cost monitored?
- What are the limitations and future improvements?

Interview note: Never claim performance improvements or production experience you cannot demonstrate.

22. HR and behavioral interview preparation

- Tell me about yourself
- Why do you want to become an AI Agent Developer?
- Explain a technical problem you solved
- Describe learning a new technology quickly
- Handling a production issue under time pressure
- Investigating a problem you cannot reproduce
- Communicating technical issues to non-technical teams
- Prioritizing incidents
- What to do when you do not know the answer
- Why should we hire you?
- Explain your project and your individual contribution
- How do you ensure AI-generated answers are trustworthy?

Interview note: Use real examples and structure behavioral answers with STAR: Situation, Task, Action, Result.

Troubleshooting method for interview scenarios

- Identify the symptom: what failed and what was expected?
- Locate the failing stage: model, API, retrieval, tool, database, or deployment.
- Collect evidence: logs, status codes, traces, inputs, outputs, and timestamps.
- Test likely causes, changing one variable at a time.
- Apply a safe fix; avoid destructive actions and verify permissions.
- Validate and prevent recurrence with regression tests, monitoring, or better error handling.

23. Project and portfolio preparation

Project 1 — IT Support RAG Agent

- Use public support tickets and a public IT knowledge base.
- Add hybrid retrieval, source citations, fallback behavior, and retrieval evaluation.
- Document dataset provenance and license.

Project 2 — Tool-Using Support Agent

- Search tickets, retrieve troubleshooting articles, classify issues, and draft responses.
- Require approval before modifying a real ticket or taking consequential actions.
- Demonstrate tool validation, logs, and error recovery.

Project 3 — Research and Report Generation Agent

- Retrieve evidence from multiple sources, verify citations, generate a report, and evaluate the result.
- Document architecture, limitations, setup steps, sample data, and measurable test results.

Prepare to explain each project

- Problem and architecture; why RAG or agents were appropriate
- Data sources, cleaning, chunking, embedding model, vector store, and retrieval
- Fallbacks, evaluation metrics, security controls, latency, and cost

- Failures encountered, root-cause analysis, limitations, and future improvements

24. What to prioritize over three months

Tier	Topics
Tier 1 — Master first	Python/debugging, REST APIs/JSON, LLM fundamentals, embeddings, RAG, vector databases, tool calling, agent architecture, troubleshooting.
Tier 2 — Prepare next	LangGraph/LangChain, agent memory, MCP, evaluation, testing, security, SQL, FastAPI, project architecture.
Tier 3 — Role-dependent	Docker, cloud deployment, advanced multi-agent systems, fine-tuning, Graph RAG, and model optimization.

Suggested 12-week plan

- Weeks 1–2: Refresh foundations, prompting, structured outputs, and LLM API/local-model basics.
- Weeks 3–4: Finish RAG ingestion, chunking, embeddings, hybrid search, citations, and evaluation.
- Weeks 5–6: Learn tool calling, state, memory, error handling, and basic LangGraph.
- Weeks 7–8: Build a practical support workflow with classification, response drafting, and human approval.
- Weeks 9–10: Add FastAPI, tests, quality measurements, and security controls.
- Weeks 11–12: Document and, if feasible, deploy projects; prepare interview answers and resume explanations.

Final interview readiness checklist

- Python and debugging
- REST APIs, HTTP, and JSON
- Git, SQL, and development tools
- LLM fundamentals and prompting
- Embeddings and vector databases
- RAG architecture and retrieval
- RAG evaluation and troubleshooting
- Agent architecture and tool calling
- Memory, state, and LangGraph
- MCP and multi-agent concepts
- Testing, observability, and security
- FastAPI, Docker, and cloud basics
- Portfolio projects and project explanations
- Technical scenarios and HR questions

Final advice: Target AI Agent Developer, Generative AI Engineer, LLM Application Developer, and Junior AI Engineer roles. Read each job description because some emphasize RAG and APIs, while others require stronger software engineering, cloud, or ML skills. Focus on being able to explain the workflow, diagnose failures, and demonstrate what you have actually built.